

MSc Individual Project Proposal

Prompts to Payloads: Inside LLM-Driven Network Attacks

Baljinnyam

Supervisors: Prof. Hamed Haddadi (h.haddadi@imperial.ac.uk), Amir Al Sadi (a.al-sadi@imperial.ac.uk)

Department of Electrical and Electronic Engineering — March 2026

1 Motivation and Research Hypotheses

Singer et al. (2024) introduced *Incalmo*, an open-source LLM-agent framework that orchestrates multi-host network attacks through a layered architecture separating high-level reasoning from low-level exploit execution. Incalmo showed that architectural scaffolding—not model scale—drives attack success. However, all evaluation was conducted in *undefended* environments: no firewalls, intrusion-detection systems (IDS), or network segmentation. This is a critical gap—real networks deploy layered defences, and we do not know how autonomous LLM attackers degrade, adapt, or fail when confronted with them.

This project tests two specific hypotheses:

H1: Deploying stateful firewalls and signature-based IDS on a multi-subnet topology reduces Incalmo’s full kill-chain completion rate by at least 50% relative to the undefended baseline.

H2: Augmenting the agent’s state representation with defence-observation signals (blocked ports, IDS alerts, connection timeouts) and equipping it with an LLM-guided replanning loop recovers at least 30% of the performance lost under the defended condition in H1.

2 Research Objectives

1. **Baseline reproduction and code audit.** Reproduce Incalmo’s results on its original scenarios (the codebase is publicly available and verified runnable). Deliver a presentation-ready walkthrough of the planner, state tracker, action translator, and tool-call interface.
2. **Defence injection and adversarial evaluation (tests H1).** Integrate configurable defences into a Docker Compose emulation environment (see Section 3). Run the unmodified Incalmo agent against four defence profiles and measure degradation across all metrics.
3. **Defence-aware state extension and adaptive replanning (tests H2).** Extend the agent’s internal state (cf. Fig. 3 of Singer et al.) with a defence-observation schema. Implement and compare two planner variants—a rule-based heuristic baseline and an LLM reflection planner—that consume the enriched state to replan around detected defences.

3 Technical Approach and Architecture

Simulation environment. All experiments run on a Docker Compose network emulation with isolated bridge networks modelling subnets. Vulnerable hosts use Metasploitable3 and custom Debian containers running known-CVE services. Defences are implemented with real tools: *iptables* rules for stateful firewalls (configured per-container via startup scripts), Docker network isolation for VLAN-equivalent segmentation, and a *Suricata* container in IDS mode running ET Open rulesets on mirrored bridge traffic. Defence profiles are parametrically toggled via YAML configuration files, ensuring full reproducibility. No traffic leaves the host machine; all containers run on an air-gapped workstation.

Defence-observation schema. The agent’s state tracker is extended with a structured record per target host: a set of observed blocked ports (populated when connection attempts return RST or timeout), a binary IDS-alert-detected flag (derived from Suricata fast.log polling), and a reachability confidence score in $[0, 1]$ decayed by repeated failures. This schema is appended to the LLM’s context at each planning step.

Planner variants. Two planners are compared: (a) a **rule-based planner** encoding expert heuristics (“if port 445 is blocked, attempt SSH lateral movement; if IDS alert detected, switch to encrypted channel or pivot through a different host”); and (b) an **LLM reflection planner** in which GPT-4o-mini is prompted with the enriched state, its last three failed actions, and a structured reflection template to propose revised attack paths. GPT-4o-mini is chosen for cost-efficiency ($\sim \$0.15/\text{M}$ input tokens); estimated API cost for the full experimental matrix (3 topologies $\times$ 4 defence profiles $\times$ 2 planners $\times$ 30 seeds = 720 runs, ~ 500 tokens/step, ~ 50 steps/run) is approximately \$270.

LLM model. The primary model is GPT-4o-mini via the OpenAI API. A sensitivity check will run a subset of conditions (medium topology, firewall+IDS, both planners, 10 seeds) on Llama-3-70B (via local inference on an A100) to assess whether findings are model-specific.

4 Evaluation Plan

Topologies. Three controlled networks: a 5-host single-subnet LAN, a 15-host multi-subnet corporate network with DMZ, and a 30-host topology with DMZ, internal, and management zones. Each is tested under four defence profiles: (i) no defence, (ii) firewall only, (iii) firewall + IDS, (iv) firewall + IDS + network segmentation.

Metrics. Attack success rate (full kill-chain completion); per-stage completion (recon, initial access, lateral movement, exfiltration); command accuracy (fraction of tool calls that are both syntactically valid and produce an expected side-effect on the target host, verified by post-action state assertions on the Docker network); adaptation efficiency (replanning cycles to first successful bypass of a defence); and wall-clock time-to-compromise.

Baselines and ablations. Baselines include: (i) vanilla Incalmo (no defence-awareness), (ii) a scripted Metasploit playbook executing a fixed attack sequence (non-LLM baseline), and (iii) AutoPentest (Henke et al., 2024) where feasible. Ablations isolate the contribution of the defence-observation schema (enriched state vs. vanilla state, same planner) and the planner type (rule-based vs. LLM reflection, same enriched state). Each cell uses 30 seeds; results report means, 95 % bootstrap confidence intervals, and Cohen’s d effect sizes. LLM stochasticity is controlled by fixing temperature to 0.0 and seeding the environment randomness only.

5 Timeline and Risk Mitigation

Weeks 1–2	Codebase audit; reproduce Incalmo baseline on original scenarios; architecture walkthrough presentation.
Weeks 3–5	Build Docker Compose environment with defence configuration module. Run unmodified agent against all defence profiles; collect H1 degradation data.
Weeks 6–8	Implement defence-observation schema and both planner variants. Run experimental matrix; collect H2 recovery data and ablation results.
Weeks 9–10	Statistical analysis; sensitivity check on Llama-3-70B subset; prepare figures and tables.
Weeks 11–12	Dissertation write-up and final submission.

Decision gate (end of Week 3): If Incalmo reproduction is incomplete, Objective 3 is reduced to the LLM reflection planner only, and topology complexity is capped at 15 hosts.

6 Expected Contributions

- **First systematic evaluation** of an LLM-driven attack agent against configurable, realistic network defences, quantifying per-defence-layer degradation of autonomous attack performance.
- **A defence-aware state extension** and replanning mechanism for the Incalmo architecture, with empirical measurement of performance recovery under defended conditions.
- **A planner comparison** (rule-based vs. LLM reflection) with ablations isolating the contribution of enriched state versus planning strategy.
- **Open, reproducible artefacts:** Docker Compose configurations, defence profiles, evaluation scripts, and all experimental data.

7 Ethics Statement

Ethical approval has been submitted under Imperial College EEE departmental review. All experiments run on isolated, air-gapped Docker environments with no internet connectivity. No real systems, live networks, or third-party infrastructure will be targeted. Released artefacts will include defence configurations and evaluation tooling but will not package ready-to-deploy offensive exploits.

8 Key References

- [1] Singer, B. et al. (2024). “Incalmo: LLM-Based Multi-Host Attack Orchestration.” arXiv:2501.16466.
- [2] Li, Z. et al. (2024). “Corba: Conquer and Divide for LLM-Based Multi-Step Reasoning in Network Penetration Testing.” GitHub: zhrli324/Corba.
- [3] Henke, J. et al. (2024). “AutoPentest: Automated Penetration Testing with LLM Agents.” GitHub: JuliusHenke/autopentest.
- [4] Fang, R. et al. (2024). “LLM Agents Can Autonomously Exploit One-Day Vulnerabilities.” arXiv:2404.08144.
- [5] Happe, A. and Cito, J. (2023). “Getting pwn’d by AI: Penetration Testing with LLMs.” ESEC/FSE 2023.